Iteration 33 – User Availability and Busy Slot Model

Resource Allocation System

May 11, 2026

1 Iteration Goal

The goal of Iteration 33 is to add the foundation for user availability in the resource allocation system. Earlier iterations focused mainly on room availability, request validation, exam allocation, multi-room exam allocation, shared-room exam allocation, and the Flask dashboard. However, the system did not yet represent when users, especially instructors or professors, are busy.

This iteration prepares the system for future committee meeting and thesis presentation scheduling. For example, a master student may need to schedule a presentation that requires three professors and one suitable room to be available at the same time. This iteration does not implement that complete meeting request yet. Instead, it adds the basic user busy-slot model, CSV loading, and availability checking service that future rules can use.

2 Changes from Previous Iteration

The previous iteration introduced a shared-room exam best-fit allocation strategy. That strategy improved physical room usage by allowing different exam requests to share the same classroom when remaining capacity is available. Iteration 33 moves to another kind of resource: human resources.

The main changes in this iteration are:

- Added the `UserBusySlot` model.
- Added the `UserAvailabilityService`.
- Added `data/user_busy_slots.csv`.
- Added `DataLoader::loadUserBusySlots(...)`.
- Added `userBusySlots` to `SystemData`.
- Updated `DataController::loadAllData(...)` to optionally load busy slots.
- Missing `user_busy_slots.csv` does not crash the program.

- Malformed busy-slot rows are skipped safely.
- Existing allocation behavior remains unchanged.

This iteration does not change the existing request types, rules, allocation strategies, or request processing workflow.

3 Scenario

The professor introduced a new resource-sharing scenario. A student or assistant may need to schedule a formal academic meeting, such as a master thesis presentation. The meeting requires several professors to attend, and the system must eventually check both room availability and participant availability.

Before this iteration, the system could check whether a room was available for a `TimeSlot`. However, it did not store whether a professor or assistant was busy during that same time. Therefore, the system needed a new foundation for representing user busy periods.

An example busy-slot file is:

```
userId,day,startTime,endTime,reason
2,1,10:00,12:00,Lecture
2,3,14:00,15:00,Office Hour
4,2,09:00,11:00,Administrative Meeting
5,4,13:00,15:00,Lab Supervision
```

This means, for example, that user 2 is busy on Monday from 10:00 to 12:00 because of a lecture.

4 Modules and Their Tasks

Module	**Task**
`UserBusySlot`	Stores one busy interval for a user, including user ID, `TimeSlot`, and reason.
`UserAvailabilityService`	Checks whether a user is available during a requested time slot.
`TimeSlot`	Represents the day and minute-based time interval used by busy slots.
`DataLoader`	Loads busy-slot rows from `user_busy_slots.csv`.
`DataController`	Coordinates loading users, spaces, requests, and optional busy-slot data.
`SystemData`	Stores the loaded `userBusySlots` vector with the rest of the system data.
Flask Dashboard	Optionally visualizes user busy slots and space allocations using existing CSV data.

Table 1: Modules added or extended in Iteration 33

5 User Availability and Busy Slot Design

The new `UserBusySlot` model represents a period when a user is not available. It contains:

- `userId`
- `TimeSlot`
- `reason`

The model is intentionally small. It only stores busy-slot data and does not perform availability logic. This follows the Single Responsibility Principle.

The `UserAvailabilityService` centralizes availability checking. It checks whether:

- the requested slot is inside working hours,
- the requested day is a working day,
- the user has an overlapping busy slot.

A user is available only if the requested slot is within the default working policy and does not overlap any busy slot for that user.

The service provides methods such as:

- `isWithinWorkingHours(slot)`
- `isUserBusy(userId, slot, busySlots)`
- `isUserAvailable(userId, slot, busySlots)`
- `getBusySlotsForUser(userId, busySlots)`

The CSV format used for busy-slot loading is:

```
userId,day,startTime,endTime,reason
```

The time values use the same minute-based time model introduced in the previous time-slot iteration. For example, `10:00` is stored internally as 600 minutes from midnight.

6 Availability Policy

The default policy assumes that users are generally available only during working hours:

- Monday to Friday
- 09:00 to 17:00
- Saturday and Sunday are unavailable

The availability service also uses the same half-open interval rule used by the scheduling model:

$$[start, end)$$

This means the start time is included, but the end time is not. Therefore, 10:00–11:00 and 11:00–12:00 do not overlap, while 10:00–11:00 and 10:30–11:30 do overlap.

Condition	**Availability Result**
Monday–Friday, 09:00–17:00, no busy overlap	Available
Overlaps with a busy slot	Unavailable
Outside 09:00–17:00	Unavailable
Saturday or Sunday	Unavailable
Adjacent to a busy slot without overlap	Available

Table 2: Default user availability policy

For example, assume the following busy slot:

```
User 2 Monday 10:00-12:00
```

Test Slot	**Result**
Monday 09:00–10:00	Available
Monday 10:00–11:00	Unavailable
Monday 11:30–12:30	Unavailable
Monday 12:00–13:00	Available
Monday 17:00–18:00	Unavailable
Saturday 10:00–11:00	Unavailable

Table 3: Example availability checks for one busy slot

7 Working Hours Policy

The working hours policy is kept inside the availability service instead of being duplicated across rules, loaders, or request classes. This keeps the design extensible. Later iterations can add custom working hours, holidays, department-specific calendars, or alternative availability policies without changing the user classes or the existing allocation strategies.

This design also keeps user subclasses simple. A `User`, `Instructor`, or `TeachingAssistant` does not decide whether it is available at a specific time. The user model still owns role and permission behavior, while the availability service owns time availability behavior.

8 Iteration Comparison

Aspect	Before Iteration 33	After Iteration 33
Room availability	Supported through allocation and availability rules	Unchanged
User availability	Not represented	Represented through `UserBusySlot`
Busy-slot CSV data	Not available	Loaded from `user_busy_slots.csv`
Availability checking	Room-focused only	User availability service added
Committee meeting support	Not implemented	Foundation prepared only
Allocation strategies	Existing strategies only	Unchanged
Request processing	Existing request flow	Unchanged

Table 4: Comparison of the system before and after Iteration 33

9 Updated Class Diagram

Figure 1 shows the updated class diagram for this iteration. The diagram should include `UserBusySlot`, `UserAvailabilityService`, `TimeSlot`, `DataLoader`, `DataController`, and `SystemData`. The important relationship is that `UserBusySlot` contains a `TimeSlot`, `DataLoader` creates busy-slot objects, `SystemData` stores them, and `UserAvailabilityService` checks availability using the loaded busy slots.

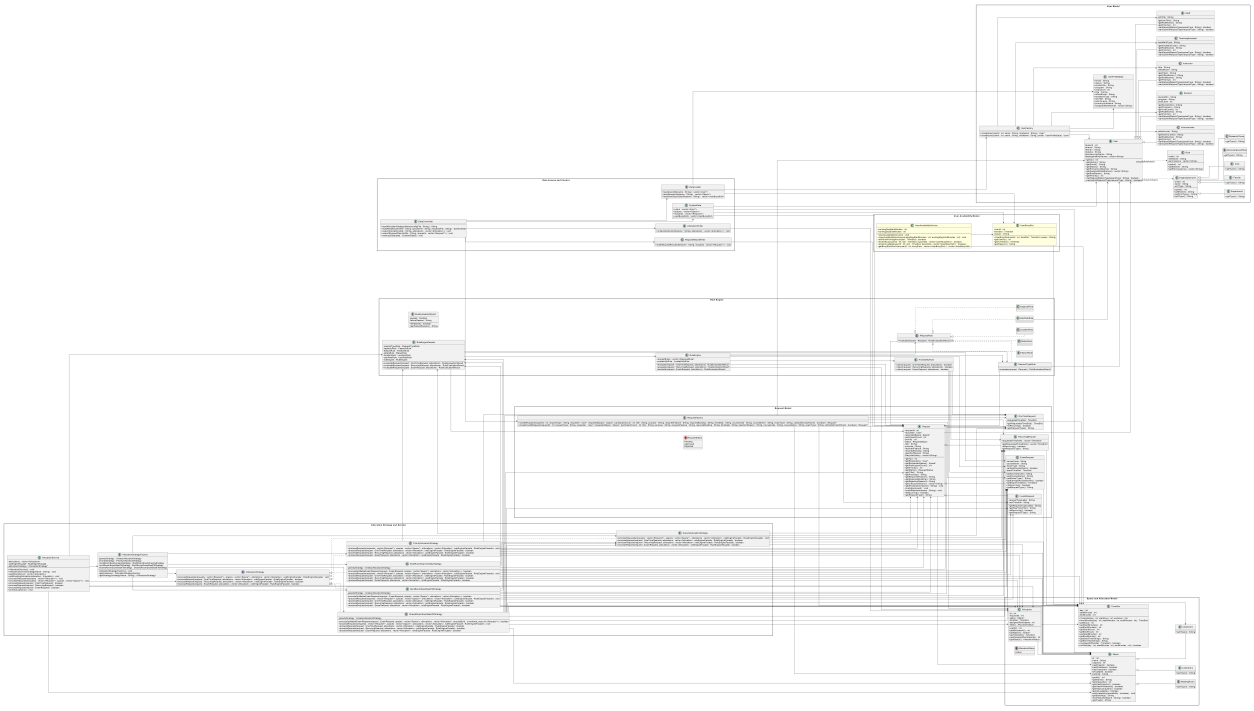

Figure 1: Updated class diagram after Iteration 33

10 Updated Workflow Diagram

Figure 2 shows the updated workflow for this iteration. The workflow remains mostly the same for normal allocation. The main addition is that `DataController` now also loads user busy slots, and the loaded data becomes available for future participant availability checks.

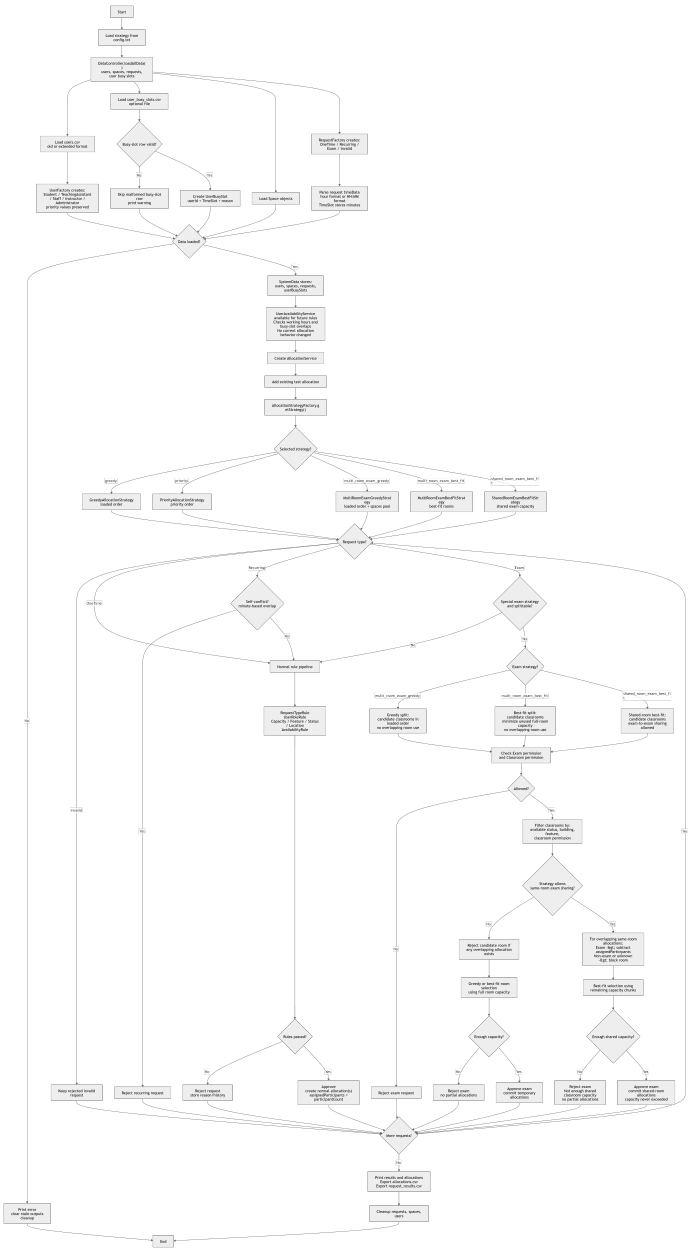

Figure 2: Updated workflow diagram after Iteration 33

11 Prototype Implementation

The prototype implementation adds the user availability foundation without changing existing allocation behavior.

The `UserBusySlot` class stores the user ID, the busy `TimeSlot`, and a textual reason. The `DataLoader` reads busy-slot data from `data/user_busy_slots.csv`. If the file is missing, the system prints a warning and continues with an empty busy-slot list. If a row is malformed, that row is skipped safely.

The `UserAvailabilityService` uses the default working-hour policy and the existing

`TimeSlot::overlapsWith(...)` behavior. It returns unavailable when a slot is outside working hours, on a weekend, or overlaps an existing busy slot.

A web-only schedule visualization was also added as a supporting dashboard feature. It reads existing CSV files only. User schedules read `user_busy_slots.csv`, while space schedules read `allocations.csv`. This dashboard view does not change the C++ backend behavior and does not implement meeting scheduling logic.

12 Summary

Iteration 33 adds the foundation for human-resource availability. The system can now load and store user busy slots and check whether a user is available during a requested time interval. This prepares the project for future committee meeting or thesis presentation scheduling, where multiple professors and a room must be available at the same time.

The iteration keeps responsibilities separated. `UserBusySlot` stores data, `DataLoader` loads CSV rows, `DataController` coordinates data loading, and `UserAvailabilityService` performs availability checks. Existing request types, rules, allocation strategies, and processing behavior remain unchanged.

This iteration is intentionally a foundation step. It does not introduce `CommitteeMeetingRequest`, participant availability rules, automatic time suggestions, or optimization. Those can be added in later iterations using the availability model introduced here.